

PERSONAL BANK ACCOUNT MANAGEMENT SYSTEM

Python Essentials – Project Report

Student Name: Daksh Shrivastava

Registration Number: 26MIM10047

Program: Integrated A.I

Semester: 1st Semester

Institution: VIT Bhopal University

Important: This is an educational simulation, not a real banking application.

1. Introduction

A command-line Python application for managing one personal account. It demonstrates functions, classes, modules, JSON storage, validation, exceptions and testing.

2. Problem Statement

Manual tracking of personal transactions can make it difficult to review balance changes and calculate totals. This application provides a simple local record system.

3. Objectives

Create a practical Python application; apply programming fundamentals; manage transactions; store data in JSON; and demonstrate modular programming and testing.

4. Functional Requirements

Account creation, PIN login, balance checking, deposits, withdrawals, simulated transfers, transaction history, financial summary, PIN change and JSON storage.

5. Non-Functional Requirements

Usability through a numbered CLI; reliability through local storage; maintainability through modules; error handling through validation and exceptions; resource efficiency through the standard library; and PIN authentication for the educational simulation.

6. Architecture

The CLI handles user interaction. Account and transaction modules implement business operations. Validation and security support safe input and authentication. Storage manages JSON persistence. Analytics calculates totals.

7. Modules

account.py, transaction.py, validation.py, security.py, storage.py, analytics.py, cli.py and tests/test_bank.py.

8. Workflow

Create account → save data → login → verify PIN → select operation → perform transaction → save changes → view history/summary → logout.

9. Implementation Details

The system uses classes, functions, conditionals, loops, exceptions, modules, JSON and unittest. It is intentionally kept understandable for a first-year student.

10. Testing

Automated tests cover deposit, withdrawal, insufficient balance, PIN verification and financial-summary calculations. Manual testing should also be performed.

11. Screenshots / Results

Add your own screenshots here before submission: main menu, account creation, deposit, withdrawal, transaction history, financial summary and unit-test output.

12. Challenges

Expected challenges include understanding modules, connecting Python files, JSON storage, input validation and exception handling.

13. Learnings

The project provides practice with object-oriented programming, functions, modules, file handling, JSON, validation, exceptions, testing and project organization.

14. Future Enhancements

SQLite storage, multiple accounts, monthly reports, GUI, CSV/PDF statements and budgeting.

15. References

Python Standard Library documentation, course materials, and the provided VITyarthi project instructions.